

Premier Health Essentials: Whop pixel and server side purchase tracking

Prepared 7 September 2026 for the Premier Health Essentials development team. Business `biz_MHH0rHJ6zrJy0D` / `premierhealthessentials.com`.

Short version. There is no Whop tracking on this site yet. This document covers the whole install in one pass: the site wide pixel that measures the top of the funnel, and the purchase event sent from your server so completed orders are never lost. Both halves are here deliberately, because retrofitting the server side afterwards means touching checkout twice.

1. What was measured

Read live on 7 September 2026 against the public site.

what	result
Hosting	Vercel
Framework	Next.js (274 <code>_next/static</code> references)
Canonical host	<code>www.premierhealthessentials.com</code> (the apex 308-redirects to www)
Whop pixel in page source	not present
<code>whop_purchase</code> in page source	not present
Whop events ever recorded	none

The last row is the one that settles it. Reading this business's own event stream back over a ten day window returns **four events, and every one of them is an account setup row** (`invites.created`, `invites.accepted`, `partners.company_affiliate.created`, and one dashboard prompt). There is not a single `pixel.page`, `view_content`, `add_to_cart` or `whop_purchase` in the account. Nothing has ever reported.

A control was run before trusting any of the above. An absurd path (`/this-path-cannot-exist-zzz9`) was requested first. It returns a genuine **404** with different content from the homepage, so this site does not serve a catch-all page for every URL and a path probe here means something. On a framework like this that check is not optional: a catch-all would return 200 for any path and make every probe look successful.

Why the page source alone would not have been enough. "Not present in page source" is weak evidence on a JavaScript rendered site, because a tag manager can inject a pixel at runtime where a source fetch will never see it. That is why the account's event stream was read as well: a pixel injected by any route would still have produced events, and there are none. The two together are conclusive where either alone would not be.

2. The site wide pixel

Two scripts, on every page, once. They record page views and issue the `_wuid` visitor cookie that section 4 depends on.

```
<!-- Site wide. Put this in the document head on EVERY page, once. -->
<script async src="https://t.whop.tw/s.js"
  data-whop-company-id="biz_MHH0rHJ6zrJy0D"></script>
<script async src="https://t.whop.tw/sc.js"></script>
```

In this codebase that belongs in the root layout rather than in a raw tag, so it loads on every route including client side navigations:

```
/* app/layout.js Next.js App Router. Loads the pixel on every route. */
/* next/script with afterInteractive keeps it out of the critical path. */
import Script from 'next/script';

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}
        <Script
          src="https://t.whop.tw/s.js"
          data-whop-company-id="biz_MHH0rHJ6zrJy0D"
          strategy="afterInteractive"
        />
        <Script src="https://t.whop.tw/sc.js" strategy="afterInteractive" />
      </body>
    </html>
  );
}
```

Do not put the company id behind an environment variable that could go missing at build time. If `data-whop-company-id` renders empty the pixel loads, reports nothing, and looks installed. That failure is silent and has cost other stores weeks.

3. The funnel events

These are browser events and belong in the browser, because they describe what the shopper is doing rather than what the business confirmed.

```

/* Fire these from the browser, where the shopper action happens. */
/* window.whop.track is available once s.js has loaded. */

/* Product page */
window.whop.track('view_content', {
  value: 129.99, currency: 'usd'
});

/* Add to cart */
window.whop.track('add_to_cart', {
  value: 129.99, currency: 'usd'
});

/* Checkout started */
window.whop.track('initiate_checkout', {
  value: 129.99, currency: 'usd'
});

```

4. The purchase event, server side

This is the one that must not come from the browser. A browser fired purchase reports a sale only when the buyer's tab reaches the confirmation page and runs the script. It reports nothing when the customer closes the tab on the payment screen, when a content blocker stops the request, or when payment confirms asynchronously minutes later. Those orders are real revenue that ad reporting never sees.

4a. The contract

```

POST https://api.whop.com/api/v1/events
Authorization: Bearer <your Whop API key>
Content-Type: application/json

{
  "event_name": "whop_purchase",
  "company_id": "biz_MHH0rHJ6zrJy0D",
  "event_id": "<stable unique id for this order>",
  "value": 129.99,
  "currency": "usd",
  "url": "https://www.premierhealthessentials.com/orders/<id>",
  "user": { "anonymous_id": "<_wuid cookie>", "email": "buyer@example.com" },
  "context": { "user_agent": "<the buyer's user agent>" }
}

```

On the **send** side the event name goes in `event_name`. Reading the same event back it returns as `pixel.custom` with the name in `custom_name`. That asymmetry is Whop's, not an error in this document.

4b. The attribution join, which is the part that is easy to miss

A server side event with no `user.anonymous_id` is a sale Whop cannot connect to the ad that produced it. It is counted as revenue and attributed to nothing.

```
/* Runs in the browser at checkout, BEFORE the order is submitted. */
/* _wuid is the visitor id the Whop pixel sets. Without it the server */
/* event cannot be joined to the ad click and the sale reports as */
/* unattributed revenue. */
function whopWuid() {
  const m = document.cookie.match(/(?:^|;\s*)_wuid=([^;]+)/);
  return m ? decodeURIComponent(m[1]) : null;
}

/* Persist it ON THE ORDER RECORD, alongside the order id, so the */
/* server still has it when the payment confirms minutes later. */
orderPayload.whop_wuid = whopWuid();
```

Capture the cookie at checkout, not at confirmation. If the payment confirms after the customer has closed the tab there is no browser left to read it from. The entire point of moving this server side is that the tab is no longer required, and reading the cookie late hands that dependency straight back.

4c. The endpoint

```
/* app/api/whop-purchase/route.js Next.js App Router on Vercel. */
/* Call this once from the code path that CONFIRMS an order, server side. */
/* Never from the browser: the customer's tab is not involved. */

/* Server env var only. Never NEXT_PUBLIC_. */
const WHOP_KEY = process.env.WHOP_API_KEY;
const COMPANY = 'biz_MHH0rHJ6zrJy0D';

export async function POST(req) {
  const order = await req.json();

  /* event_id is what makes this safe to call more than once. Whop */
  /* de-duplicates on it, so a retry, a refresh or a replayed webhook */
  /* cannot double count the same order. */
  const body = {
    event_name: 'whop_purchase',
    company_id: COMPANY,
    event_id: 'purchase_' + order.id,
    value: Number(order.total),
    currency: 'usd',
    url: 'https://www.premierhealthessentials.com/orders/' + order.id,
    user: {
      anonymous_id: order.whop_wuid || undefined,
      email: order.email || undefined
    },
    context: { user_agent: order.user_agent || undefined }
  };

  const r = await fetch('https://api.whop.com/api/v1/events', {
    method: 'POST',
    headers: {
      'Authorization': 'Bearer ' + WHOP_KEY,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(body)
  });

  if (!r.ok) {
    /* Log it and retry out of band. Do NOT fail the customer's order */
    /* because a tracking call failed. */
    console.error('whop purchase post failed', r.status, await r.text());
  }
  return new Response(null, { status: 204 });
}
```

Keep the API key server side. On Vercel that means a plain environment variable, not one prefixed `NEXT_PUBLIC_`. A key exposed to the browser lets anyone write events into this account.

5. How to confirm it works

Place one real order, then read the account back:

```
curl -s -H "Authorization: Bearer $WHOP_API_KEY" \\  
  "https://api.whop.com/api/v1/events?company_id=biz_MHH0rHJ6zrJy0D&from=2026-09-  
07T00:00:00Z&to=2026-09-08T00:00:00Z"
```

Look for a row with `custom_name: "whop_purchase"` carrying the order's real total in `total_usd_amount`. Two further checks separate a real server side install from a browser one:

1. **Send it twice.** Post the same `event_id` again. The event count must not go up. If it does, the idempotency key is not derived from anything stable.
2. **Kill the tab.** Complete a checkout and close the browser before the confirmation page renders. The purchase event must still arrive. That is the whole reason for the work, and it is the one test a browser install cannot pass.

Whop's event stream lags ingestion by a few minutes, and an empty read taken too soon is not evidence of failure. If a window looks empty, widen it to several days before concluding anything. A single empty response has caused more than one false alarm on this platform.